MARRAKECH
جامعة القاضي عياض
UNIVERSITÉ CADI AYYAD

Semlalia
Faculté des Sciences Semlalia
Département de L'informatique

Python pour l'Intelligence Artificielle
Chapitre 4 : Scikit-learn

Licence IA SD - S5

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

scikit-learn

- **scikit-learn** est l'une des **bibliothèques Python** les plus populaires pour l'**apprentissage automatique (machine learning).** Elle offre un large éventail d'**algorithmes d'apprentissage supervisé** et **non supervisé**, ainsi que des outils pour la **préparation** des **données**, la **sélection** de **modèle**, l'**évaluation** de **performances** et bien plus encore.

La bibliothèque python pour le machine learning

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Principales fonctionnalités et caractéristiques

1. **Facilité d'utilisation :** scikit-learn est conçu pour être simple à utiliser et à comprendre. Sa syntaxe claire et cohérente permet aux utilisateurs de rapidement mettre en œuvre des algorithmes d'apprentissage automatique sans avoir besoin de connaissances avancées en mathématiques ou en informatique.
2. **Large sélection d'algorithmes :** scikit-learn offre une gamme complète d'algorithmes pour les tâches d'apprentissage supervisé (classification, régression) et non supervisé (clustering, réduction de dimensionnalité). Cela inclut des méthodes populaires telles que les k-plus proches voisins, les arbres de décision, les machines à vecteurs de support (SVM), les forêts aléatoires, la régression linéaire, etc.
3. **Intégration avec NumPy et SciPy :** scikit-learn est construit sur NumPy et SciPy, deux bibliothèques fondamentales pour le calcul scientifique en Python. Cela lui permet de tirer parti de leurs structures de données et de leurs fonctions pour une manipulation efficace des données et une implémentation optimisée des algorithmes.
4. **Outils de prétraitement des données :** scikit-learn propose une variété d'outils pour la préparation des données, y compris le traitement des valeurs manquantes, le codage des variables catégorielles, la normalisation des caractéristiques, etc. Ces outils sont essentiels pour préparer les données avant de les utiliser pour l'apprentissage automatique.

Principales fonctionnalités et caractéristiques

5. **Évaluation des modèles :** scikit-learn fournit des métriques d'évaluation de modèle telles que la précision, le rappel, la F1-score, l'aire sous la courbe ROC, etc. Ces métriques permettent d'évaluer les performances des modèles sur des ensembles de données de test et d'effectuer des comparaisons entre différents modèles.
6. **Validation croisée :** scikit-learn propose des techniques de validation croisée telles que la validation croisée k-fold, qui permettent d'estimer la performance d'un modèle de manière robuste en utilisant l'ensemble des données disponibles.
7. **Sélection de modèle :** scikit-learn offre des outils pour la sélection de modèle, y compris la recherche sur grille (grid search) et la recherche aléatoire (random search), qui permettent d'identifier les hyperparamètres optimaux pour un modèle donné.
8. **Extensibilité :** scikit-learn est conçu pour être extensible, ce qui signifie que vous pouvez facilement ajouter de nouveaux algorithmes ou étendre les fonctionnalités existantes en créant vos propres classes et fonctions.

Principales fonctionnalités et caractéristiques

9. **Documentation exhaustive :** Scikit-learn est bien documenté avec des exemples détaillés et une documentation complète. Cela en fait une ressource précieuse pour les personnes qui souhaitent apprendre les bases de l'apprentissage automatique ou explorer de nouveaux concepts.
10. **Support de la communauté :** Scikit-learn bénéficie d'une communauté active qui contribue au développement de la bibliothèque et fournit un support via des forums en ligne, des listes de diffusion, et des canaux de communication sur les réseaux sociaux.

Liste des principales sous bibliothèque de scikit-learn

Scikit-learn est composé de plusieurs **sous-bibliothèques**, chacune offrant des fonctionnalités spécifiques pour différentes tâches d'apprentissage automatique. Voici une liste des principales sous-bibliothèques de scikit-learn:

1. **sklearn.datasets :** Cette sous-bibliothèque contient des jeux de données de démonstration et des utilitaires pour charger des ensembles de données externes. Elle offre des fonctions pour charger des ensembles de données populaires tels que Iris, Boston Housing, et digits.
2. **sklearn.preprocessing :** Utilisé pour effectuer des prétraitements sur les données avant de les utiliser pour l'apprentissage automatique. Il offre des fonctionnalités telles que la normalisation, la binarisation, l'imputation des valeurs manquantes, etc.
3. **sklearn.feature_extraction :** Contient des utilitaires pour extraire des caractéristiques à partir de données brutes. Cela inclut la transformation de texte en vecteurs numériques à l'aide de méthodes telles que TF-IDF et le sac de mots.
4. **sklearn.feature_selection :** Cette sous-bibliothèque fournit des outils pour sélectionner les caractéristiques les plus pertinentes pour l'apprentissage automatique. Elle comprend des méthodes de sélection basées sur les scores, la récursivité, etc.

Liste des principales sous bibliothèque de scikit-learn

6. **sklearn.model_selection :** Utilisé pour diviser les données en ensembles d'entraînement et de test, ainsi que pour effectuer une validation croisée afin d'évaluer les performances des modèles.
7. **sklearn.metrics :** Contient une large gamme de métriques d'évaluation de modèle telles que l'accuracy, la précision, le rappel, le F1-score, l'aire sous la courbe ROC, etc.
8. **sklearn.linear_model :** Fournit des modèles de régression linéaire, de régression logistique et d'autres méthodes de régression.
9. **sklearn.svm :** Contient des implémentations des machines à vecteurs de support (SVM) pour la classification et la régression.
10. **sklearn.tree :** Contient des implémentations d'arbres de décision pour la classification et la régression

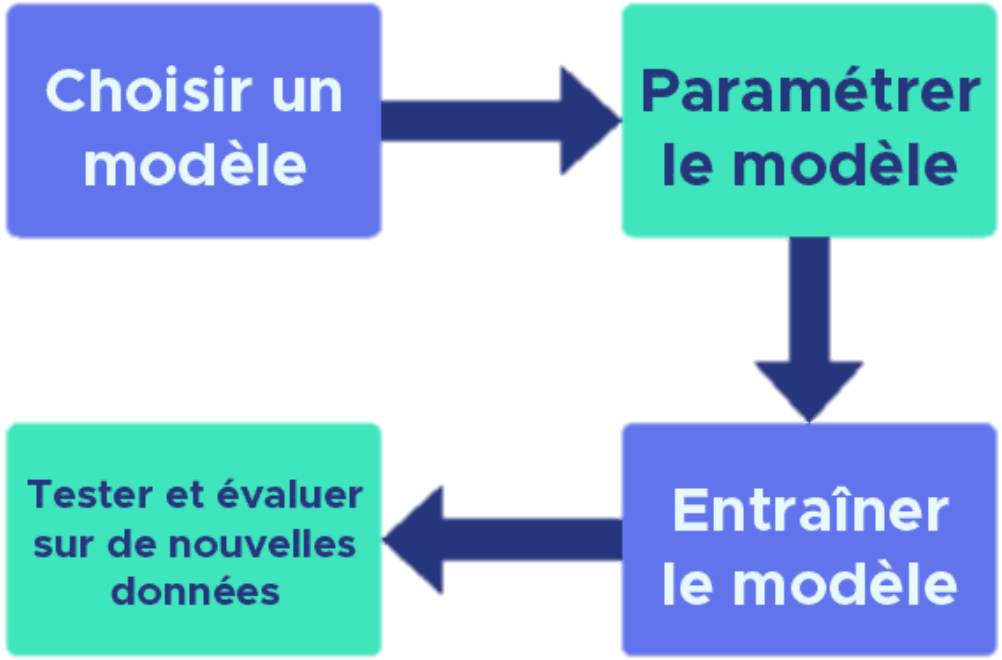
Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Liste des principales sous bibliothèque de scikit-learn

10. **sklearn.ensemble :** Cette sous-bibliothèque fournit des méthodes d'ensemble telles que les forêts aléatoires, les méthodes de boosting (AdaBoost, Gradient Boosting), etc.
11. **sklearn.cluster :** Utilisé pour effectuer le clustering des données à l'aide d'algorithmes tels que KMeans, DBSCAN, etc.
12. **sklearn.naive_bayes :** Contient des implémentations de classificateurs naïfs de Bayes, tels que MultinomialNB et GaussianNB.
13. **sklearn.neighbors :** Fournit des méthodes pour le calcul des voisins les plus proches, utilisé dans des algorithmes tels que KNeighborsClassifier et KNeighborsRegressor.
14. **sklearn.neural_network :** Contient des implémentations de réseaux de neurones pour la classification, la régression et le clustering.
15. **sklearn.pipeline :** Permet de chaîner plusieurs étapes de traitement des données et de modélisation en une seule étape, facilitant ainsi le déploiement et la réutilisation des pipelines d'apprentissage automatique.

Les bases de Scikit-Learn

- Comme l'API est uniformisée, Quel que soit le modèle que vous envisagez d'utiliser, les étapes de modélisation sont souvent les mêmes :



Les bases de Scikit-Learn

- **Choisir un modèle** en important la classe appropriée de Scikit-Learn.
- **Paramétrer le modèle**. Si vous êtes déjà sûrs des paramètres que vous voulez utiliser vous pouvez les renseigner à la main sinon la librairie offre aussi des techniques comme GridSearchCV pour trouver les paramètres optimaux.
- **Entraîner le modèle** sur le jeu d'apprentissage à l'aide de la méthode fit.
- **Tester le modèle** sur de nouvelles données :
  - Dans le cadre d'apprentissage supervisé, nous utilisons la méthode **predict** sur les données test.
  - Dans le cadre d'apprentissage non supervisé, nous utilisons les méthodes **transform** ou **predict**.

Ces 4 étapes sont en général communes à l'utilisation d'un grand nombre de modèles disponibles dans la librairie ce qui permet, une fois que vous avez compris la logique de construction d'un modèle, de pouvoir très facilement utiliser d'autres modèles.

Concepts fondamentaux de Scikit-learn

- L'API de `Sklearn` s'articule autour de trois concepts clés :
  - **Estimateurs**
  - **Transformateurs**
  - **Prédicteurs**
- Un estimateur est un objet qui apprend à partir de données (par exemple, `un classificateur`).
- Les transformateurs traitent les données à l'avance (par exemple, `mise à l'échelle des caractéristiques`).
- Les prédicteurs fournissent des prédictions (par ex, `predict()`),
- Le flux de travail suit un schéma **ajustement-transformation-prévision** :

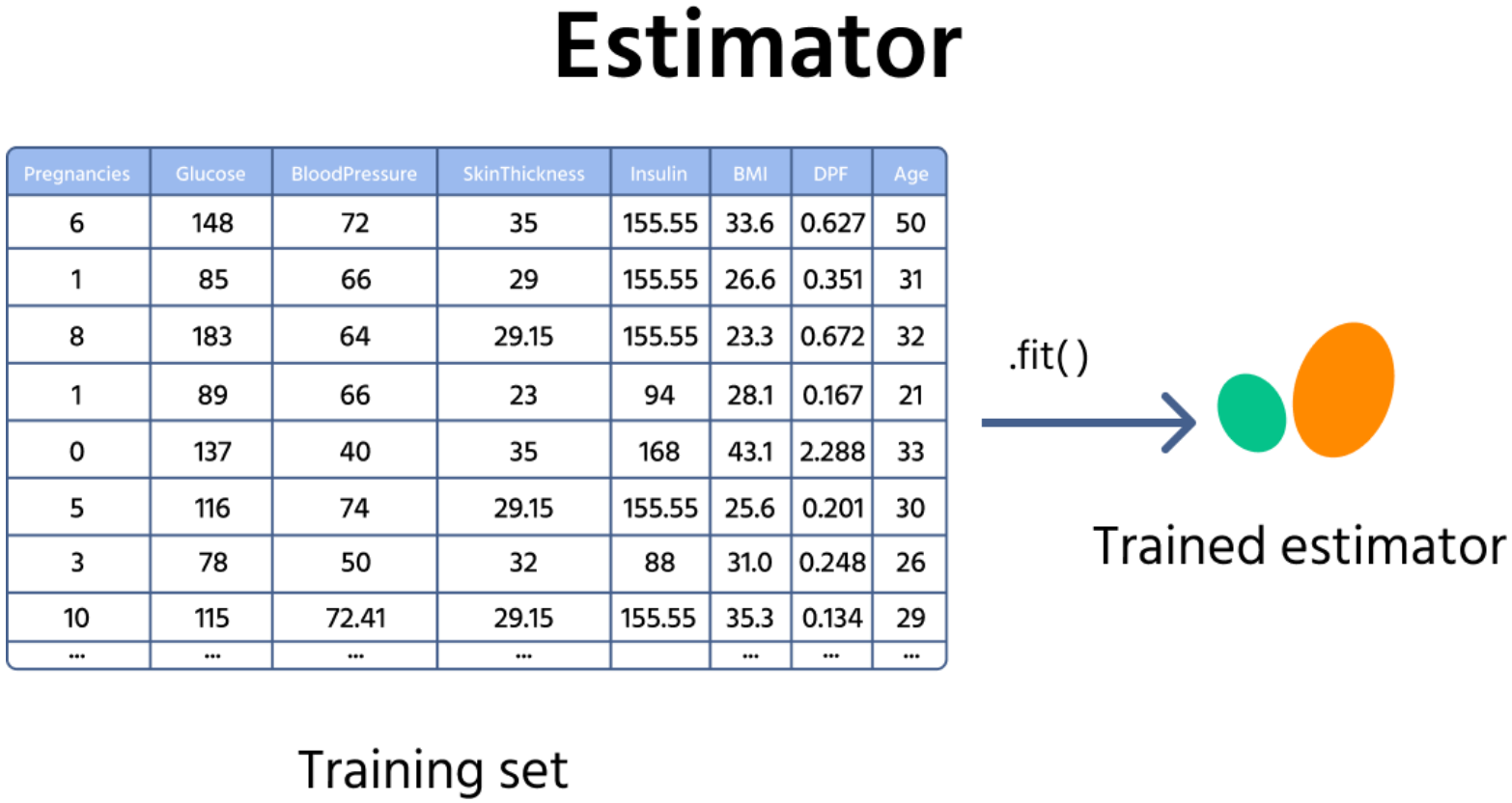
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Concepts fondamentaux de Scikit-learn

Estimateurs (Estimators)

- Un **Estimateur** est **tout objet capable d'apprendre à partir de données**.
  C'est la base de l'API sklearn : presque toutes les classes sont des estimateurs.

```
estimator.fit(X, y)      # supervised
estimator.fit(X)         # unsupervised
```

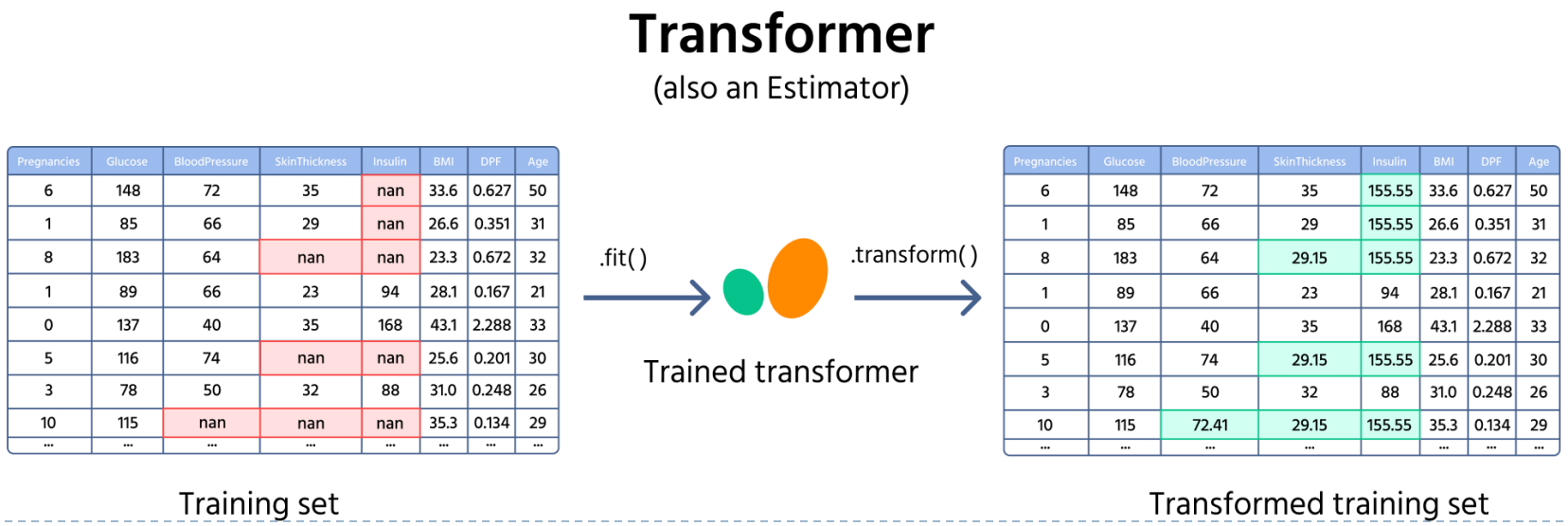


Concepts fondamentaux de Scikit-learn

Transformateurs

- Un **Transformateur** est un type particulier d'estimateur qui **modifie les données**.
- Un transformateur possède `.fit()` et `.transform()`, ainsi que `.fit_transform()` pour effectuer les deux opérations en une seule fois.

```
.fit(X, y=None)          # apprendre les paramètres de transformation
.transform(X)            # appliquer la transformation
.fit_transform(X)        # forme combinée
```



Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DPF	Age
6	148	72	35	nan	33.6	0.627	50
1	85	66	29	nan	26.6	0.351	31
8	183	64	nan	nan	23.3	0.672	32
1	89	66	23	94	28.1	0.167	21
0	137	40	35	168	43.1	2.288	33
5	116	74	nan	nan	25.6	0.201	30
3	78	50	32	88	31.0	0.248	26
10	115	nan	nan	nan	35.3	0.134	29
...	...	...	...	...	...	...	...

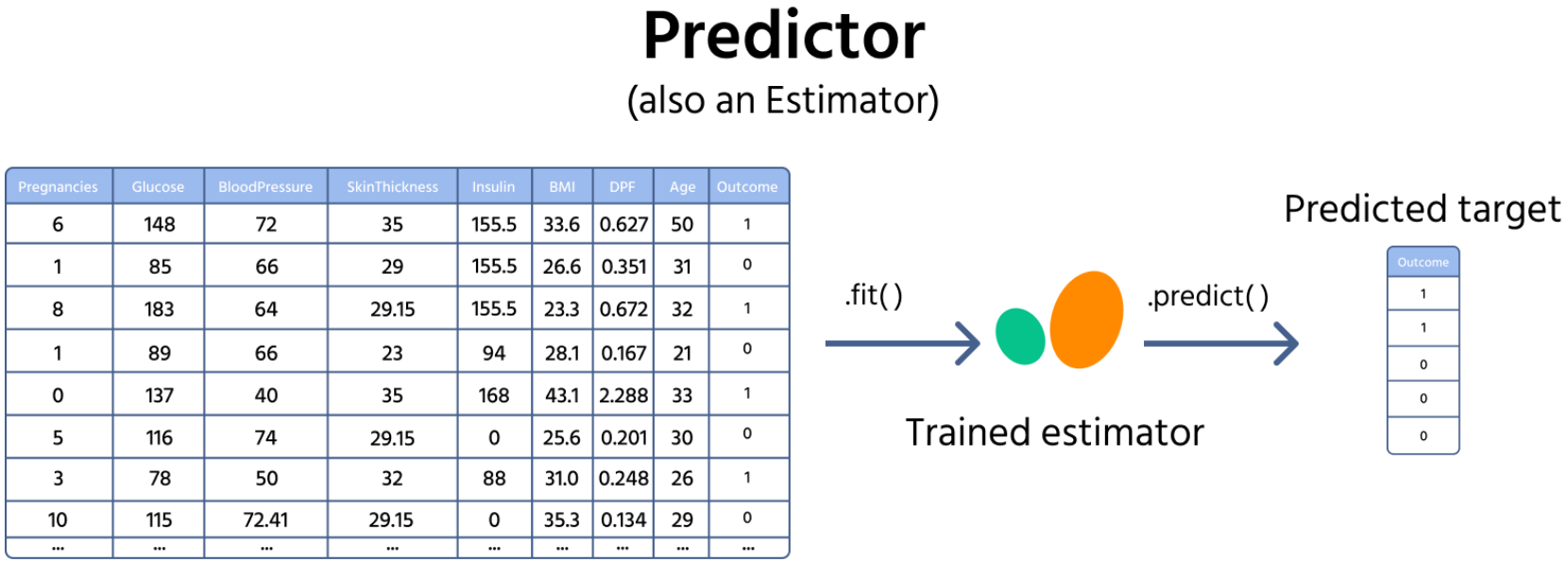
Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DPF	Age
6	148	72	35	155.55	33.6	0.627	50
1	85	66	29	155.55	26.6	0.351	31
8	183	64	29.15	155.55	23.3	0.672	32
1	89	66	23	94	28.1	0.167	21
0	137	40	35	168	43.1	2.288	33
5	116	74	29.15	155.55	25.6	0.201	30
3	78	50	32	88	31.0	0.248	26
10	115	72.41	29.15	155.55	35.3	0.134	29
...	...	...	...		...	...	...

Concepts fondamentaux de Scikit-learn

Prédicteurs

- Un **Prédicteur** est un estimateur **capable de faire des prédictions** après avoir appris.
- Un prédicteur est un estimateur doté de `.predict()` pour générer des sorties.

```
predictor.fit(X, y)
predictor.predict(X_new)
```

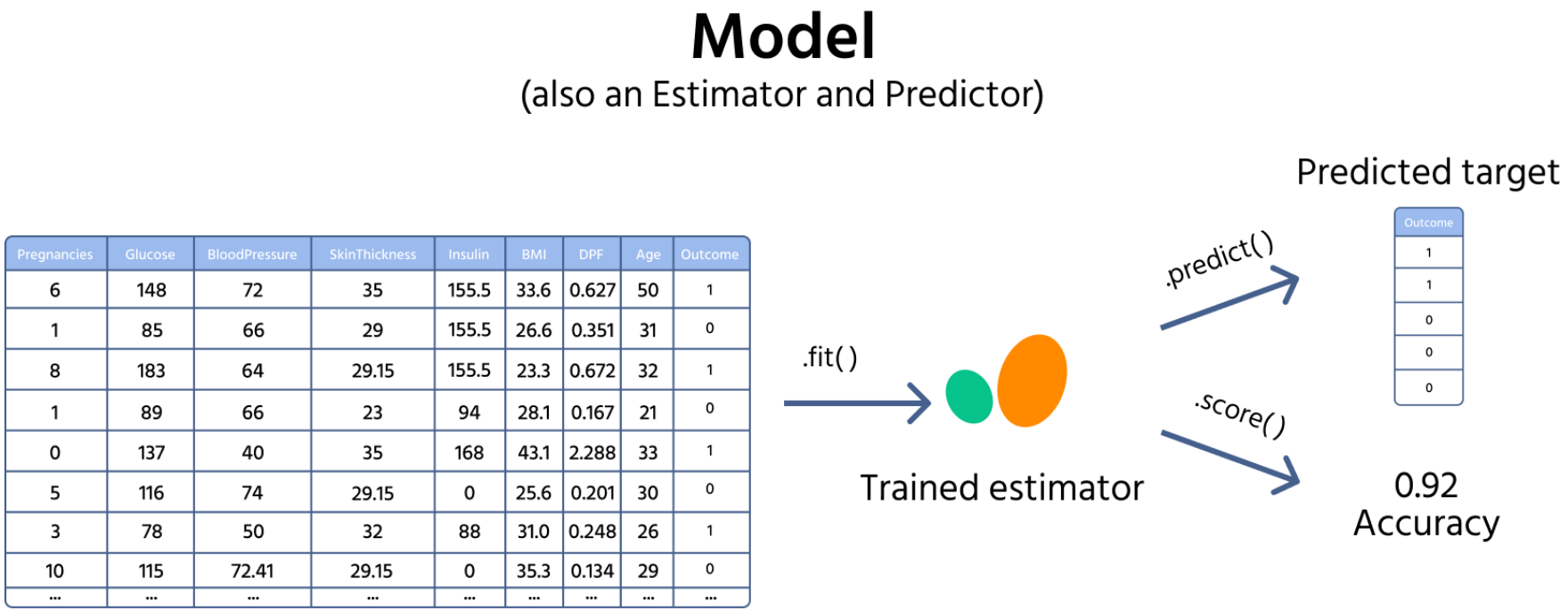


Concepts fondamentaux de Scikit-learn

Modèle

- Un modèle est un prédicteur doté de `.score()`, qui évalue la performance.

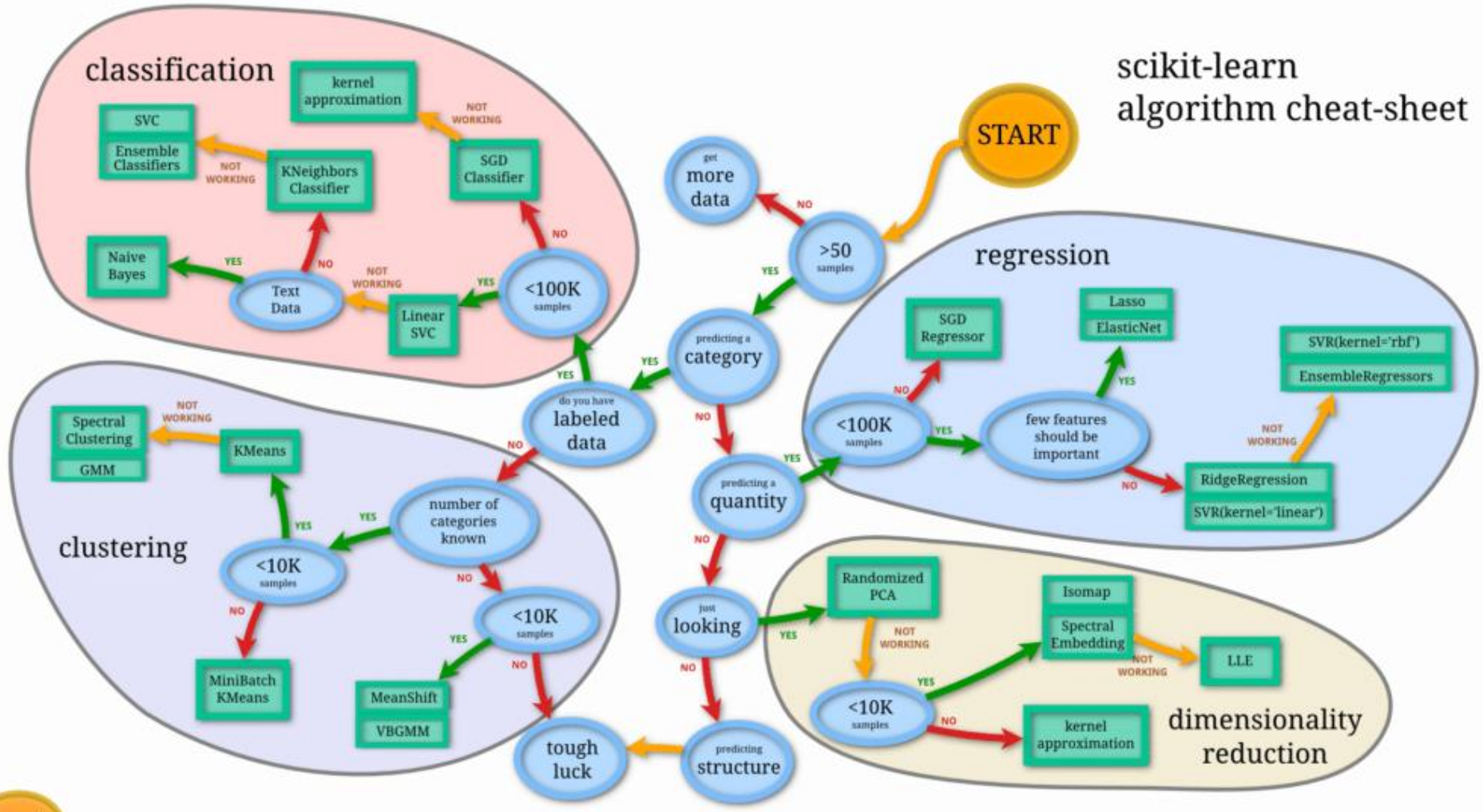
```
model.fit(X, y)
model.score(X, y)
```



Concepts fondamentaux de Scikit-learn

Type de classe	Méthodes requises
Estimateur	`.fit()`
Transformateur	`.fit()`, `.transform()`, `.fit_transform()`
Prédicteur	`.fit()`, `.predict()`
Modèle	`.fit()`, `.predict()`, `.score()`

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma



La carte décisionnelle (flowchart)

Choisir le bon algorithme Scikit-Learn selon :

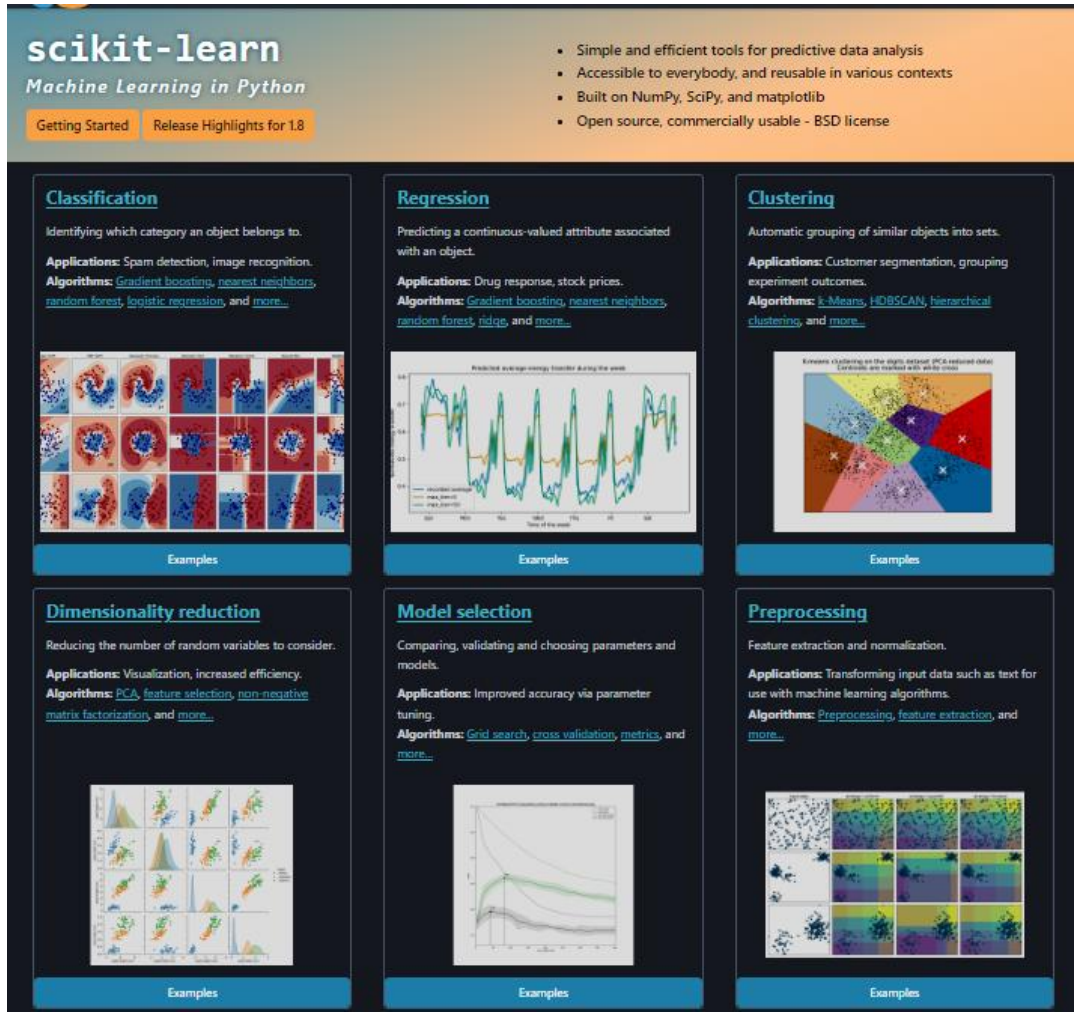
- le type de problème
- la quantité de données
- si les données sont étiquetées ou non
- les caractéristiques du dataset

Concepts fondamentaux de Scikit-learn

Sur le site de documentation, les fonctions sont réparties en **6 thèmes** :

- Classification
- Régression
- Clustering
- Dimensionality reduction
- Model selection
- Preprocessing

La documentation est très complète et contient de nombreux exemples facilement réutilisables, n'hésitez pas à y faire quelques recherches !



Prétraitement des Données avec Scikit-learn

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Imputation des Valeurs Manquantes

- La classe `SimpleImputer` remplace automatiquement les valeurs manquantes.

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer()
```

- Ses paramètres principaux :
  - `missing_value` : valeur considérée comme manquante (par défaut `np.nan`) ;
  - `strategy` : méthode de remplissage des lacunes (`'mean'` par défaut) ;
  - `fill_value` : utilisé lorsque `strategy='constant'`.
- En tant que **transformateur**, il fournit des méthodes telles que `.fit()`, `.transform()` et `.fit_transform()`.

```
imputer = SimpleImputer(strategy='mean')
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Imputation des Valeurs Manquantes

Méthode	Description
`fit(X)`	Ajuste les données
`transform(X)`	Renvoie X transformé
`fit_transform(X)`	Identique à `fit(X).transform(X)`
`set_output()`	Définit le type de sortie

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Imputation des Valeurs Manquantes

- SimpleImputer attend un DataFrame, et **non** une Series. Un DataFrame à une seule colonne doit être sélectionné avec une double paire de crochets :

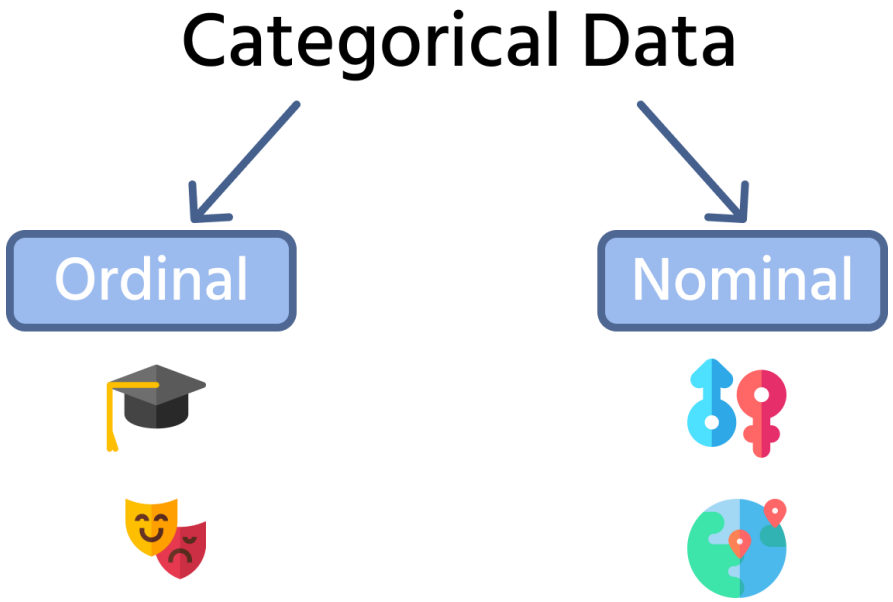
```
imputer.fit_transform(df[['column']])
```

- fit_transform() renvoie un **tableau 2D**, mais l'affectation à une colonne de DataFrame nécessite un tableau **1D**. Aplatissez le résultat en utilisant .ravel() :

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

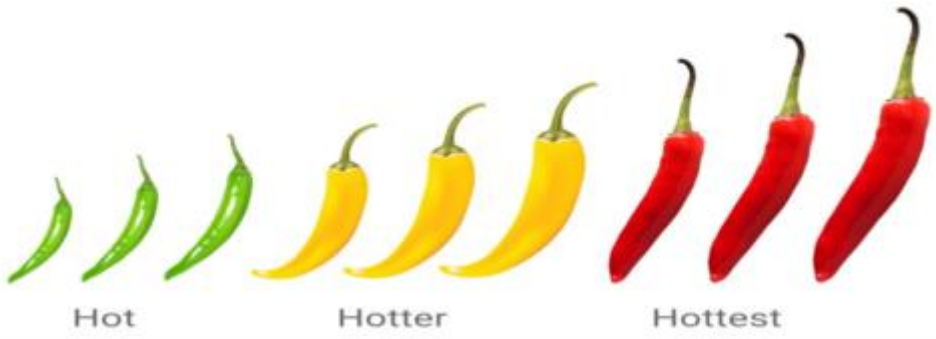
Encodage des variables catégorielles

- L'encodage des variables catégorielles consiste à les convertir en un format numérique que les algorithmes des Machine Learning peuvent comprendre. Les méthodes les plus courantes sont :
  - **Tri de données ordinales**
  - **Encodage one-hot**
  - **Encodage de label**



Encodeur Ordinal

- Le tri des données ordinales consiste à organiser les points de données en fonction de leurs valeurs ordinales ou de leurs catégories dans un ordre spécifique.
- Les données ordinales représentent des catégories ayant un ordre naturel ou un classement, comme faible, moyen, élevé ou petit, moyen ou grand.



Encodeur Ordinal

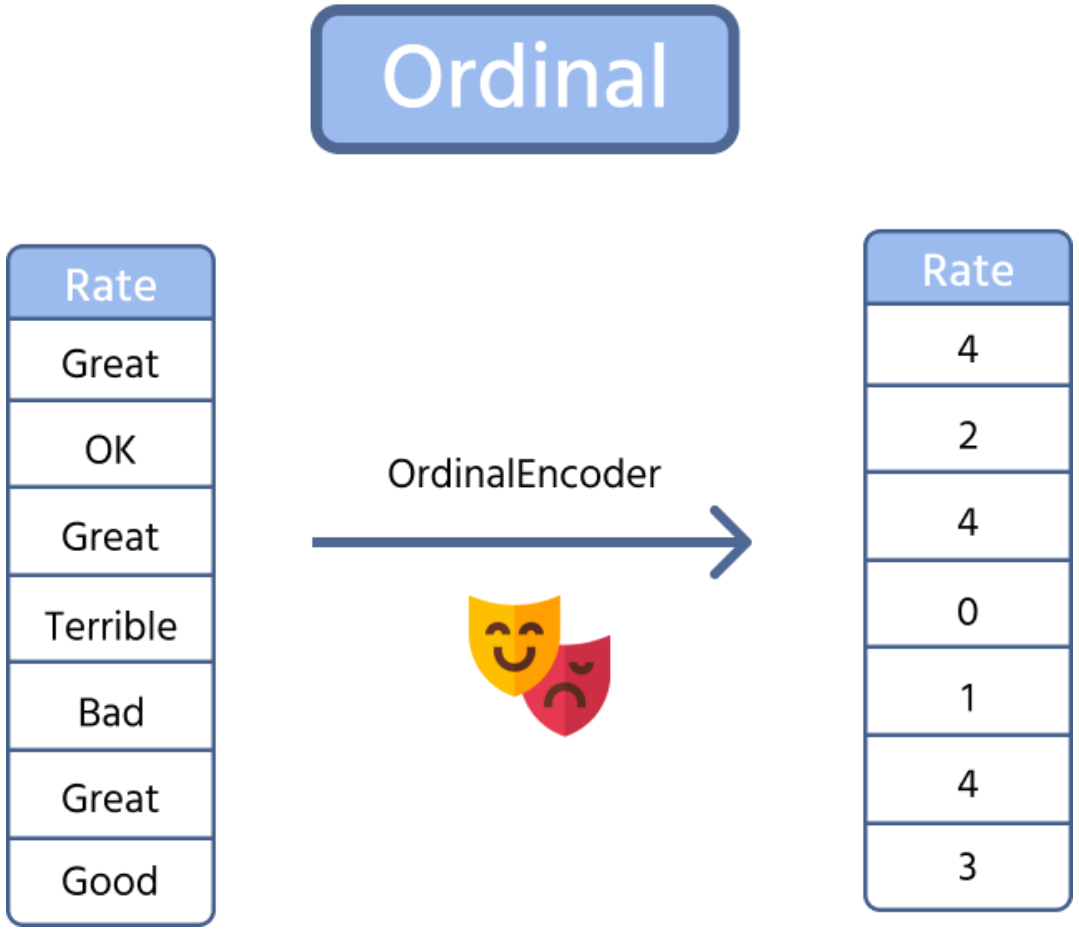
- Les données ordinales possèdent un ordre naturel, tandis que les données nominales n'en ont pas. En raison de cet ordre, les catégories peuvent être encodées sous forme de nombres selon leur classement.
- Par exemple, une colonne 'rate' avec les valeurs 'Terrible', 'Bad', 'OK', 'Good' et 'Great' peut être encodée ainsi :
  - 'Terrible' → 0
  - 'Bad' → 1
  - 'OK' → 2
  - 'Good' → 3
  - 'Great' → 4
- Pour encoder des données ordinales (variables avec un ordre) en **nombres entiers**., on utilise le OrdinalEncoder. Celui-ci convertit les catégories en entiers à partir de 0.

Encodeur Ordinal

```
sklearn.preprocessing.OrdinalEncoder

Constructor:
 ● OrdinalEncoder(categories='auto')
   categories – ordered categories for each column.
   Default value is 'auto' to determine automatically

Methods:
 ● fit(X) – Fit the data
 ● transform(X) – Returns the transformed X
 ● fit_transform(X) – Same as .fit(X).transform(X)
 ● set_output() – Set the output type
 ● inverse_transform(X) – Convert back to the original
representation
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodeur Ordinal



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodeur Ordinal

- Lors de la transformation de plusieurs caractéristiques avec OrdinalEncoder, les catégories pour chaque colonne doivent être spécifiquement définies. Cela se fait via l'argument categories :

```
encoder = OrdinalEncoder(categories=[col1_categories, col2_categories, ...])
```

```
from sklearn.preprocessing import OrdinalEncoder
import numpy as np
X = np.array([
    ["Mauvais"],
    ["Bon"],
    ["Très mauvais"],
    ["Excellent"]
])
encoder = OrdinalEncoder()
X_encoded = encoder.fit_transform(X)
print(X_encoded)
```

```
[[2.]
 [0.]
 [3.]
 [1.]]
```

L'ordre est **automatique** selon l'ordre alphabétique par défaut :
`['Excellent', 'Bon', 'Mauvais', 'Très mauvais']`

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodeur Ordinal

```
from sklearn.preprocessing import OrdinalEncoder
import numpy as np
X = np.array([
    ["Mauvais"],
    ["Bon"],
    ["Très mauvais"],
    ["Excellent"]
])
encoder = OrdinalEncoder(categories=[[
    "Très mauvais",
    "Mauvais",
    "Moyen",
    "Bon",
    "Excellent"
]])
X_encoded = encoder.fit_transform(X)
print(X_encoded)
```

```
[[1.]
 [3.]
 [0.]
 [4.]]
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodeur One-Hot

- Pour les données ordinales, telles que les évaluations d'utilisateurs allant de 'Terrible' à 'Excellent', les encoder sous forme de nombres de 0 à 4 est approprié car le modèle peut saisir l'ordre inhérent.
- En revanche, pour une caractéristique comme 'city' comportant cinq catégories distinctes, les encoder sous forme de nombres de 0 à 4 suggérerait à tort un ordre. Dans ce cas, l'encodage one-hot est préférable, car il représente les catégories sans impliquer de hiérarchie.
- Pour encoder des données nominales, on utilise le transformateur OneHotEncoder. **Il crée une colonne pour chaque valeur unique**. Ensuite, pour chaque ligne, il attribue 1 à la colonne correspondant à la valeur de cette ligne et 0 aux autres colonnes.

Nominal

City
NewYork
LosAngeles
NewYork
Chicago
NewYork
Chicago
LosAngeles

OneHotEncoder →

City_NewYork	City_LosAngeles	City_Chicago
1	0	0
0	1	0
1	0	0
0	0	1
1	0	0
0	0	1
0	1	0

Ce qui était à l'origine 'NewYork' possède désormais **1** dans la colonne 'City_NewYork' et **0** dans les autres colonnes City_.

Encodeur One-Hot

```
sklearn.preprocessing.OneHotEncoder

Constructor:
 ● OneHotEncoder()

Methods:
 ● fit(X) – Fit the data
 ● transform(X) – Returns the transformed X
 ● fit_transform(X) – Same as .fit(X).transform(X)
 ● set_output() – Set the output type
 ● inverse_transform(X) – Convert back to the original
representation
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodeur One-Hot

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder

X = np.array([["Rouge"], ["Bleu"], ["Vert"], ["Rouge"]])

encoder = OneHotEncoder(sparse_output=False)
X_encoded = encoder.fit_transform(X)
print(X_encoded)
```

```
[[0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]]
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodeur d'Étiquettes

- Les OrdinalEncoder et OneHotEncoder sont généralement utilisés pour encoder les caractéristiques (la variable X). Cependant, la variable cible (y) peut également être catégorielle.
- LabelEncoder est un outil de Scikit-Learn utilisé pour **convertir les étiquettes (cibles) catégorielles** en nombres entiers.
- Il sert principalement à **encoder la variable cible y** dans un problème de classification.

```
y = ["chat", "chien", "poisson", "chat"]
```

chat → 0
chien → 1
poisson → 2
chat → 0

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodeur d'Étiquettes

```
from sklearn.preprocessing import LabelEncoder

y = ["Rouge", "Bleu", "Vert", "Rouge"]

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

print(y_encoded)
```

```
[2 0 1 2]
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Encodage des variables catégorielles

Ordinal

Rate
Great
OK
Great
Terrible
Bad
Great
Good

OrdinalEncoder →

Rate
4
2
4
0
1
4
3

Nominal

City
NewYork
LosAngeles
NewYork
Chicago
NewYork
Chicago
LosAngeles

OneHotEncoder →

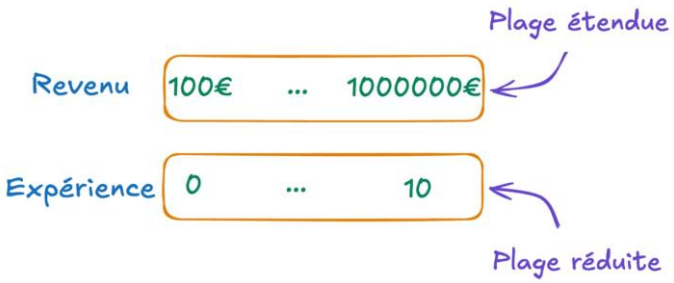
City_NewYork	City_LosAngeles	City_Chicago
1	0	0
0	1	0
1	0	0
0	0	1
1	0	0
0	0	1
0	1	0

Colonne	Encodeur
X, valeurs ordinales	OrdinalEncoder
X, valeurs nominales	OneHotEncoder
y	LabelEncoder

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques

- Après avoir traité les valeurs manquantes et encodé les variables catégorielles, l'ensemble de données ne présente plus de problèmes susceptibles de provoquer des erreurs dans le modèle. Cependant, un autre défi subsiste : des échelles de caractéristiques différentes.
- Ce problème ne provoquera pas d'erreurs si les données dans leur état actuel sont utilisées dans le modèle, mais il peut **détériorer considérablement certains modèles d'apprentissage automatique**.



La mise à l'échelle des caractéristiques

- Considérer un exemple où une caractéristique est 'age', variant de **18** à **50**, et une seconde caractéristique est 'income', variant de **$25,000** à **$500,000**. Il est évident qu'une différence de dix ans d'âge est plus significative qu'une différence de dix dollars de revenu.
- Cependant, certains modèles, tels que **k-NN**, peuvent considérer ces différences comme **d'importance équivalente**. Par conséquent, la colonne 'income' aura un impact bien plus important sur le modèle. Il est donc essentiel que les caractéristiques aient **approximativement la même plage de valeurs** pour que k-NN fonctionne efficacement.
- Bien que d'autres modèles soient moins sensibles aux différences d'échelle, la mise à l'échelle des données peut **accélérer considérablement le traitement**. Ainsi, la normalisation des données est généralement incluse comme étape finale du prétraitement.

La mise à l'échelle des caractéristiques

Objectif :

- **Eliminer l'effet d'unité qui pourrait donner mécaniquement plus de poids à un facteur qu'à un autre.**
- Imaginons que vous voulez analyser les facteurs qui influencent le prix d'une maison. Vous avez deux variables principales :
  - Surface habitable : mesurée en **m²** (valeurs typiques : 80, 120, 200 m²)
  - Distance au centre-ville : mesurée en **km** (valeurs typiques : 2, 5, 15 km)
- Maison A: Surface = 100 m², Distance = 5 km → Point (100, 5)
- Maison B: Surface = 120 m², Distance = 3 km → Point (120, 3)
- La distance euclidienne entre ces deux maisons serait :

$$\sqrt{(120-100)^2+(3-5)^2} \approx 20{,}1$$

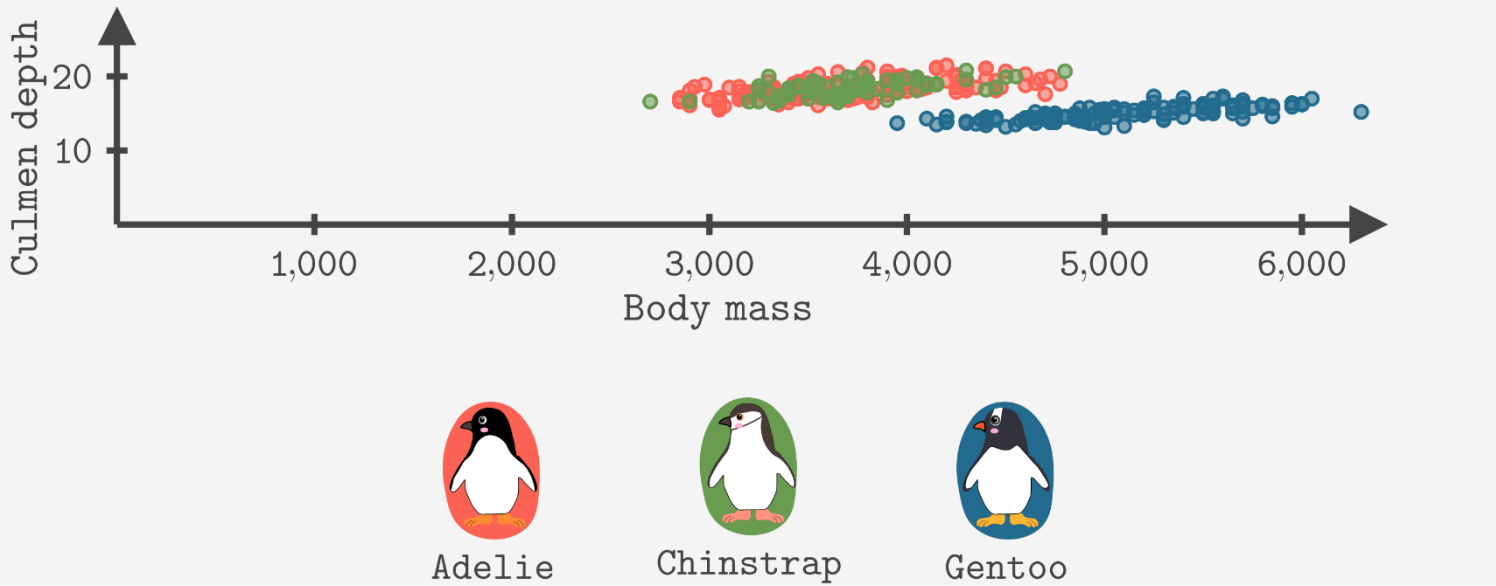
- La différence de surface (20 m²) domine complètement la différence de distance (2 km) simplement parce que les m² sont numériquement plus grands que les km dans cet exemple.

La mise à l'échelle des caractéristiques

- La mise à l'échelle des caractéristiques ajuste l'étendue des données sans fausser les différences entre les valeurs.
- Il existe plusieurs techniques de mise à l'échelle, la normalisation et la standardisation étant les plus couramment utilisées.
- Il existe trois approches populaires pour la mise à l'échelle des données :
  - `MinMaxScaler` : met à l'échelle les caractéristiques dans une plage de `[0, 1]` ;
  - `MaxAbsScaler` : met à l'échelle les caractéristiques de sorte que la valeur absolue maximale soit 1 (ainsi, les données sont garanties d'être dans une plage de `[-1, 1])` ;
  - `StandardScaler` : standardise les caractéristiques en rendant la moyenne égale à `0` et la variance égale à `1`.

La mise à l'échelle des caractéristiques

- Pour illustrer le fonctionnement des scalers, considérez les caractéristiques 'culmen_depth_mm' et 'body_mass_g' du jeu de données penguins. Ces caractéristiques peuvent être représentées graphiquement pour observer leurs échelles.



	species	island	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	18.7	181.0	3750.0	MALE
1	Adelie	Torgersen	17.4	186.0	3800.0	FEMALE
2	Adelie	Torgersen	18.0	195.0	3250.0	FEMALE
3	Adelie	Torgersen	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	19.3	193.0	3450.0	FEMALE

La mise à l'échelle des caractéristiques

MinMaxScaler

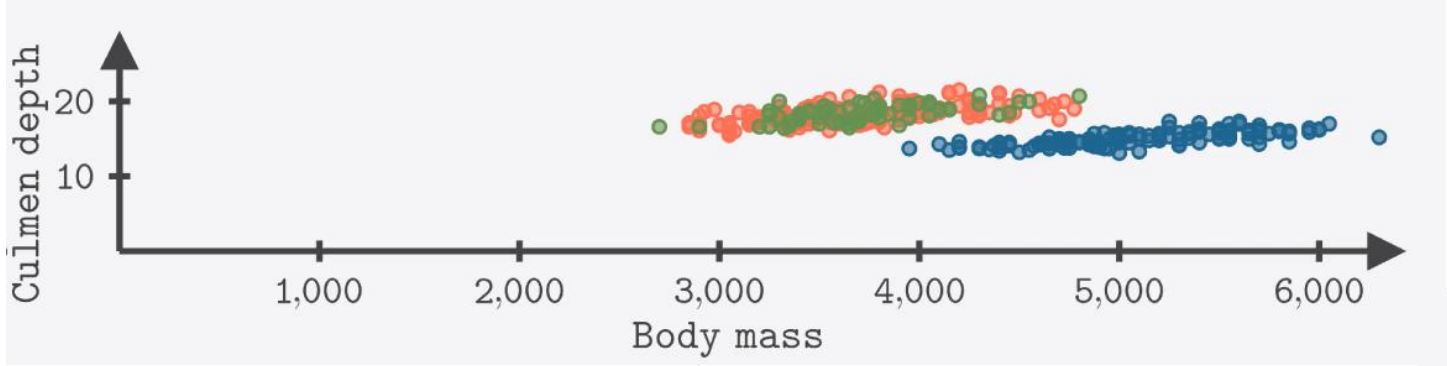
- Le MinMaxScaler fonctionne en soustrayant la valeur minimale (pour que les valeurs commencent à zéro) puis en divisant par (x_{max} - x_{min}) afin que le résultat soit inférieur ou égal à 1.

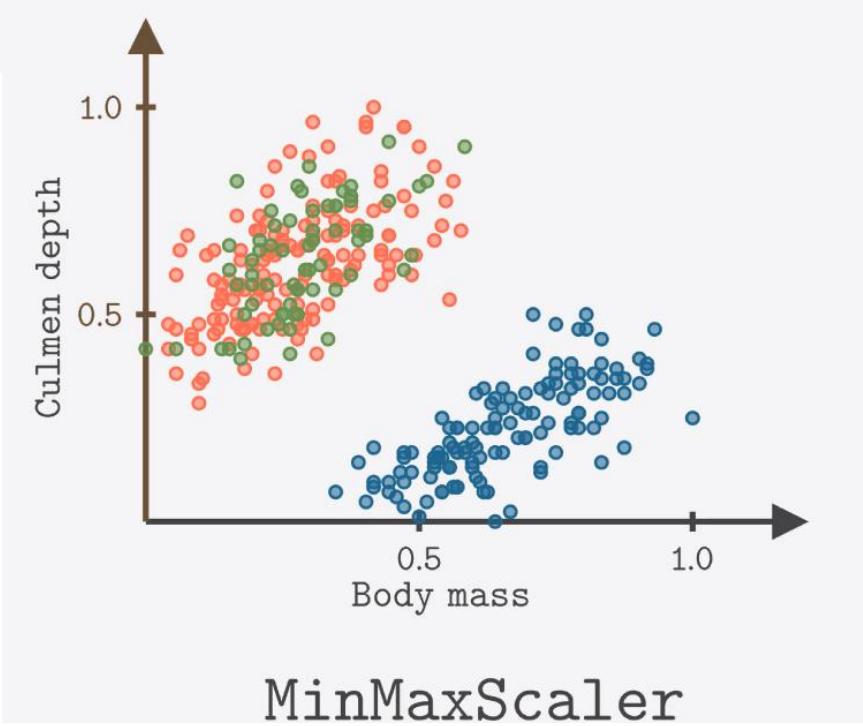
$$\text{MinMaxScaler}: \; x_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

```
sklearn.preprocessing.MinMaxScaler

Constructor:
 ● MinMaxScaler()

Methods:
 ● fit(X) – Fit the data
 ● transform(X) – Returns the transformed X
 ● fit_transform(X) – Same as .fit(X).transform(X)
 ● set_output() – Set the output type
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques





Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques

MaxAbsScaler

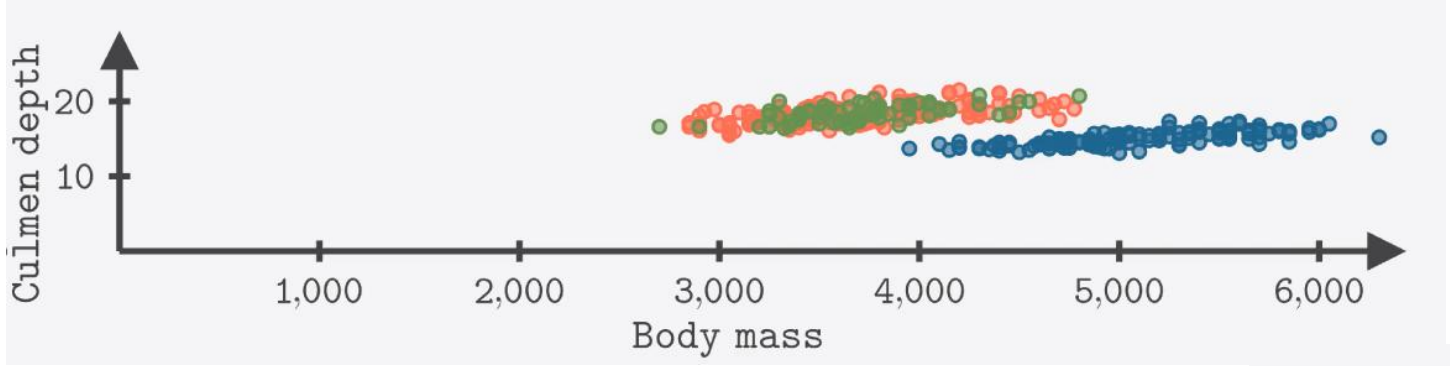
- Le MaxAbsScaler fonctionne en recherchant la **valeur absolue maximale** et en **divisant** chaque valeur par celle-ci. Cela garantit que la valeur absolue maximale est égale à 1.

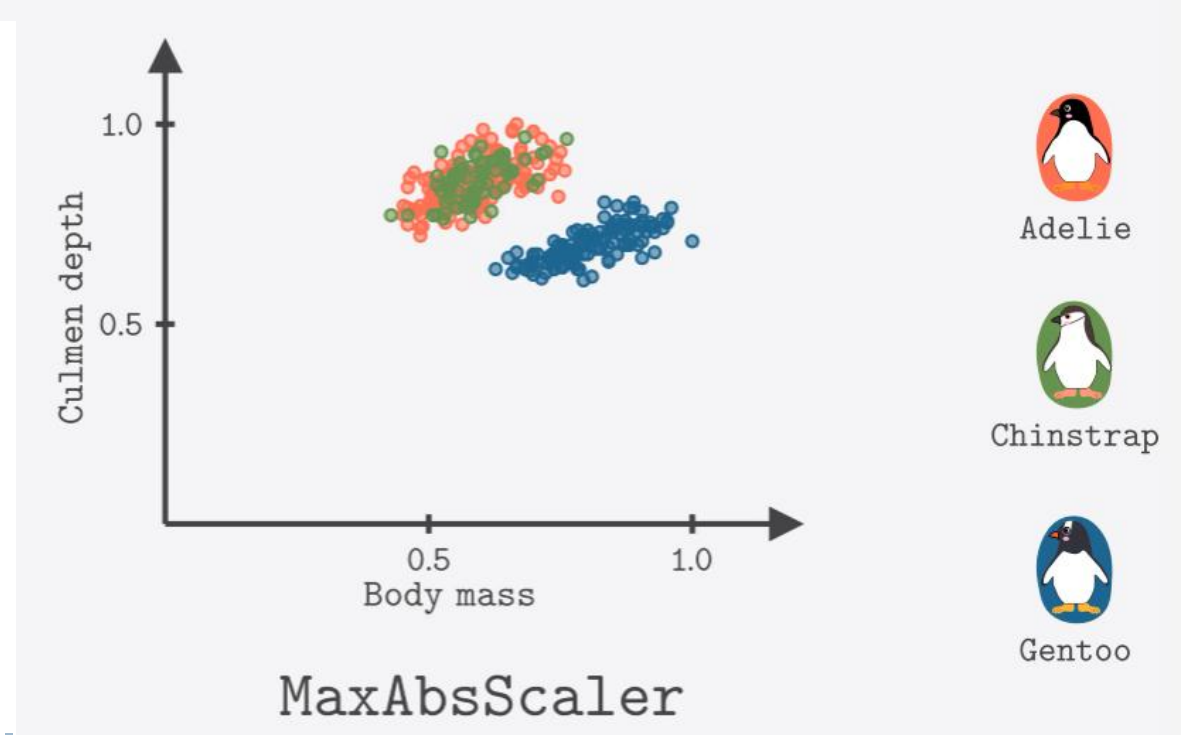
$$\text{MaxAbsScaler}: \; x_{\text{scaled}} = \frac{x}{\max|x|}$$

```
sklearn.preprocessing.MaxAbsScaler

Constructor:
 ● MaxAbsScaler()

Methods:
 ● fit(X) – Fit the data
 ● transform(X) – Returns the transformed X
 ● fit_transform(X) – Same as .fit(X).transform(X)
 ● set_output() – Set the output type
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques





Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques

StandardScaler

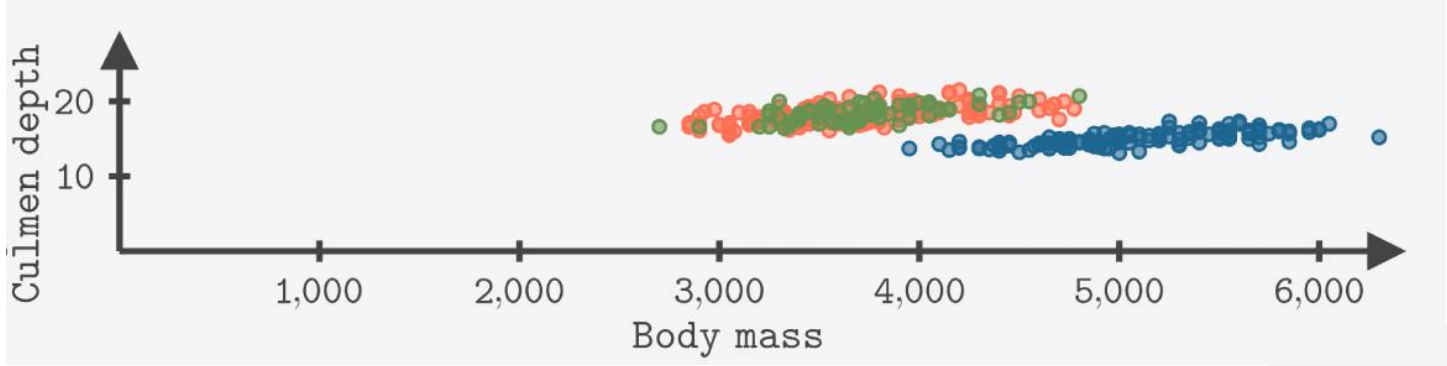
- Le principe du StandardScaler provient des statistiques. Il fonctionne en soustrayant la **moyenne** (pour centrer autour de zéro) et en divisant par l'**écart type** (pour que la variance soit égale à 1).

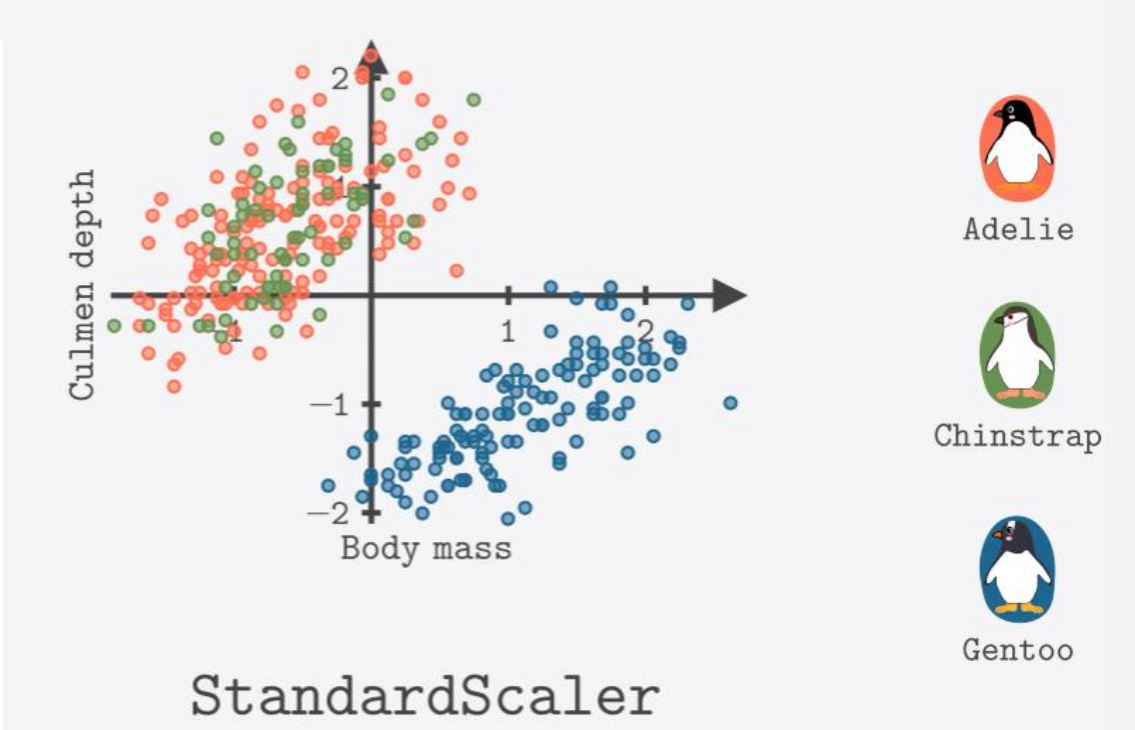
$$\text{StandardScaler}: \; x_{\text{scaled}} = \frac{x - \bar{x}}{\sigma}$$

```
sklearn.preprocessing.StandardScaler

Constructor:
 ● StandardScaler()

Methods:
 ● fit(X) – Fit the data
 ● transform(X) – Returns the transformed X
 ● fit_transform(X) – Same as .fit(X).transform(X)
 ● set_output() – Set the output type
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques





Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques

- Voici un exemple de code avec MinMaxScaler. Les autres scalers s'appliquent de la même manière.

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

X = np.array([
    [1, 200],
    [5, 300],
    [10, 400]
])

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
print(X_scaled)
```

```
[[0.         0.        ]
 [0.44444444 0.5       ]
 [1.         1.        ]]
```

- Il faut séparer train et test
- Ne jamais faire fit sur le test → data leakage.

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

La mise à l'échelle des caractéristiques

Quel scaler utiliser ?

- Un `StandardScaler` est plus **sensible aux valeurs aberrantes**, ce qui le rend moins adapté comme scaler par défaut. Si vous préférez une alternative à StandardScaler, le choix entre `MinMaxScaler` et `MaxAbsScaler` dépend de la préférence personnelle, selon que vous souhaitez mettre à l'échelle les données dans l'intervalle `[0,1]` avec `MinMaxScaler` ou dans l'intervalle `[-1,1]` avec `MaxAbsScaler.`

La mise à l'échelle des caractéristiques

Robuste

- La standardisation classique utilise la moyenne et l'écart-type, mais elle est sensible aux valeurs extrêmes (outliers).
- La standardisation robuste est moins sensible aux outliers, car elle utilise la médiane et l'IQR, qui sont plus stables.
- Le procédé est le suivant: on soustrait aux valeurs de la variable la médiane et on divise par l'écart interquartile IQR la formule est la suivante:

$$X_{rob} = \frac{X - mediane}{IQR}$$

```python
import numpy as np
from sklearn.preprocessing import RobustScaler

X = np.array([[1], [2], [2], [3], [100]])

scaler = RobustScaler()
X_robust = scaler.fit_transform(X)

print(X_robust.flatten())
```

```
[-1.  0.  0.  1. 98.]
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Pipelines

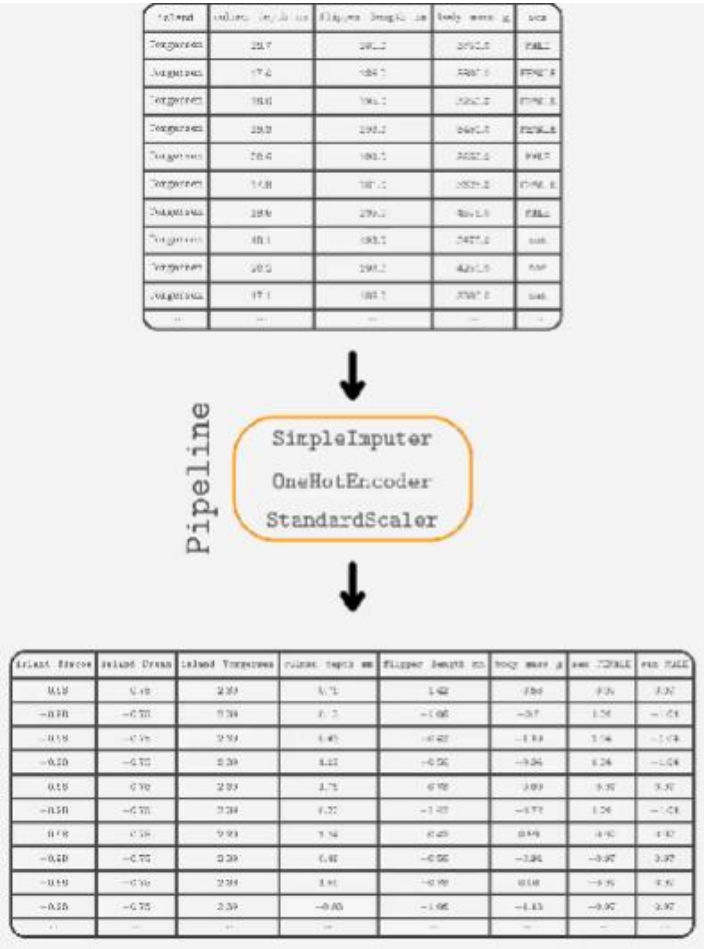
Pipelines

- Un pipeline est une méthode élégante pour combiner toutes les étapes de prétraitement ainsi qu'un modèle. Les pipelines facilitent grandement l'entraînement et l'utilisation d'un modèle.
- Dans la section précédente, trois étapes de prétraitement ont été réalisées : imputation, encodage et mise à l'échelle.
- Les étapes de prétraitement ont été appliquées **une par une**, en transformant des colonnes spécifiques et en les réintégrant dans le tableau X. Cette méthode peut s'avérer fastidieuse, en particulier avec OneHotEncoder, qui modifie le nombre de colonnes.
- Un autre inconvénient est que toute nouvelle donnée utilisée pour la prédiction doit suivre la même séquence de transformations, ce qui nécessite de répéter l'ensemble du processus.
- La classe Pipeline de Scikit-learn simplifie cela en combinant toutes les transformations dans un flux de travail unique, facilitant ainsi l'application cohérente du prétraitement aux données d'entraînement comme aux nouvelles instances.

Pipelines

Une **Pipeline** est un outil de Scikit-Learn qui permet de **chaîner plusieurs étapes du traitement des données** :

- nettoyage
- imputation
- encodage
- normalisation
- entraînement du modèle



Pipelines

- Un Pipeline agit comme un conteneur pour une séquence de transformateurs, et éventuellement un estimateur. Lorsque la méthode .fit_transform() est appelée sur un Pipeline, elle applique **séquentiellement** la méthode .fit_transform() de chaque transformateur aux données.
- Cette approche rationalisée signifie qu'il suffit d'appeler .fit_transform() **une seule fois sur l'ensemble d'entraînement** puis d'utiliser la méthode .transform() pour traiter de nouvelles instances.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

# Create a pipeline with three steps: imputation, one-hot encoding, and scaling
pipeline = Pipeline([
  ('imputer', SimpleImputer(strategy='most_frequent')), # Step 1: Impute missing values
  ('encoder', OneHotEncoder()),                         # Step 2: Convert categorical data
  ('scaler', StandardScaler())                          # Step 3: Scale the data
])

# Fit and transform the data using the pipeline
X_transformed = pipeline.fit_transform(X)
```

Transformateur de Colonnes

- Lors de l'appel à .fit_transform(X) sur un Pipeline, chaque transformateur est appliqué à **toutes les colonnes**, ce qui n'est pas toujours souhaitable.
- Certaines colonnes peuvent nécessiter des encodeurs différents — par exemple, OrdinalEncoder pour les variables ordinales et OneHotEncoder pour les variables nominales.
- ColumnTransformer résout ce problème en permettant d'assigner **différents transformateurs à des colonnes spécifiques** à l'aide de make_column_transformer.

```
make_column_transformer(*transformers, remainder='drop')


Arguments:
 ● *transformers – tuples of form (transformer, columns)
 ● remainder – action for unspecified columns,
'drop'/'passthrough'

Returns:
 ● ct – a ColumnTransformer object
```

Transformateur de Colonnes

- make_column_transformer accepte des tuples de `(transformer, [columns])`. Par exemple, appliquer OrdinalEncoder à 'education' et OneHotEncoder à 'gender'

```
ct = make_column_transformer(
        (OrdinalEncoder(), ['education']),
        (OneHotEncoder(), ['gender']),
        remainder='passthrough'
)
```

- remainder contrôle le traitement des colonnes non spécifiées. Par défaut : 'drop'. Pour conserver toutes les autres colonnes inchangées, définir remainder='passthrough'.

Transformateur de Colonnes

- Par exemple, considérez le fichier exams.csv. Il contient plusieurs colonnes **nominales** ('gender', 'race/ethnicity', 'lunch', 'test preparation course') et une colonne **ordinale**, 'parental level of education'.

```
   gender race/ethnicity parental level of education         lunch  \
0    male        group D          associate's degree      standard
1    male        group C            some high school      standard
2  female        group D            some high school  free/reduced
3    male        group D            some high school      standard
4    male        group C                some college      standard

  test preparation course  math score  reading score  writing score
0                    none          79             78             72
1               completed          81             73             71
2                    none          68             75             79
3                    none          66             49             46
4               completed          65             67             66
```

Transformateur de Colonnes

- En utilisant ColumnTransformer, les données nominales peuvent être transformées avec OneHotEncoder et les données ordinales avec OrdinalEncoder **en une seule étape**

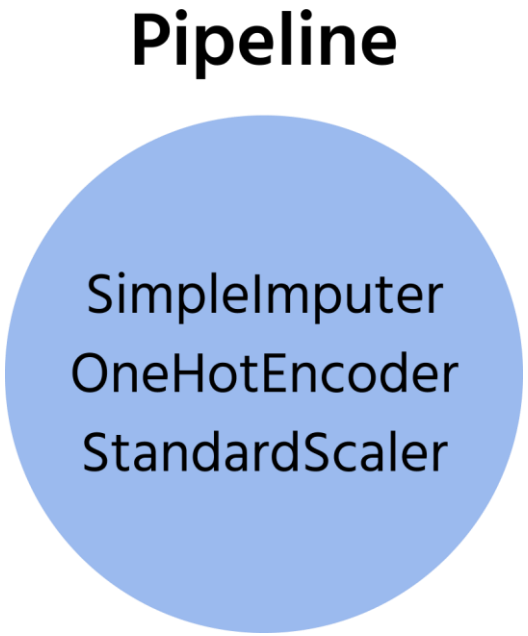
```
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

edu_categories = ['high school', 'some high school', 'some college', "associate's degree",
"bachelor's degree", "master's degree"]

ct = make_column_transformer(
(OrdinalEncoder(categories=[edu_categories]), ['parental level of education']),
(OneHotEncoder(), ['gender', 'race/ethnicity', 'lunch', 'test preparation course']),
remainder='passthrough'
)

print(ct.fit_transform(df))
```

```
[[ 3.  0.  1. ... 79. 78. 72.]
 [ 1.  0.  1. ... 81. 73. 71.]
 [ 1.  1.  0. ... 68. 75. 79.]
 ...
 [ 4.  1.  0. ... 54. 59. 68.]
 [ 2.  0.  1. ... 79. 75. 73.]
 [ 4.  1.  0. ... 79. 82. 89.]]
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Prétraitement Efficace des Données avec des Pipelines

- Avec la possibilité de transformer les colonnes séparément à l'aide de la fonction make_column_transformer, l'étape suivante consiste à construire des pipelines. Un **pipeline** est un conteneur qui organise les étapes de prétraitement et les applique **séquentiellement**.



Prétraitement Efficace des Données avec des Pipelines

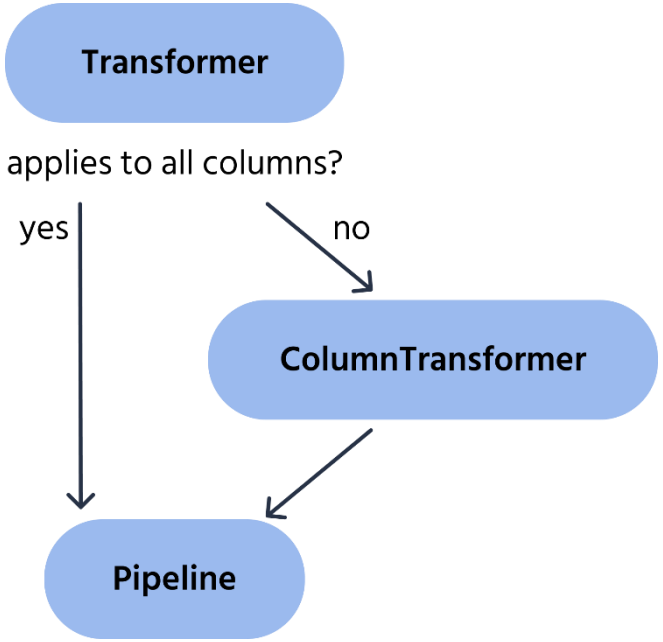
- Un pipeline dans Scikit-learn peut être créé soit à l'aide du constructeur de la classe Pipeline, soit avec la fonction make_pipeline du module sklearn.pipeline.

```
sklearn.pipeline.make_pipeline(*steps)

Arguments:
 ● *steps – any number of transformers + final estimator

Returns:
 ● p – a Pipeline object
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Prétraitement Efficace des Données avec des Pipelines

- Il suffit de passer tous les **transformateurs** en arguments à une fonction. La création de pipelines est aussi simple que cela.
- Cependant, lorsque vous appelez la méthode .fit_transform(X) sur l'objet Pipeline, elle applique .fit_transform(X) à **chaque transformateur à l'intérieur du pipeline**. Ainsi, si vous souhaitez traiter certaines colonnes différemment, il convient d'utiliser un ColumnTransformer et de le passer à make_pipeline().



Prétraitement Efficace des Données avec des Pipelines

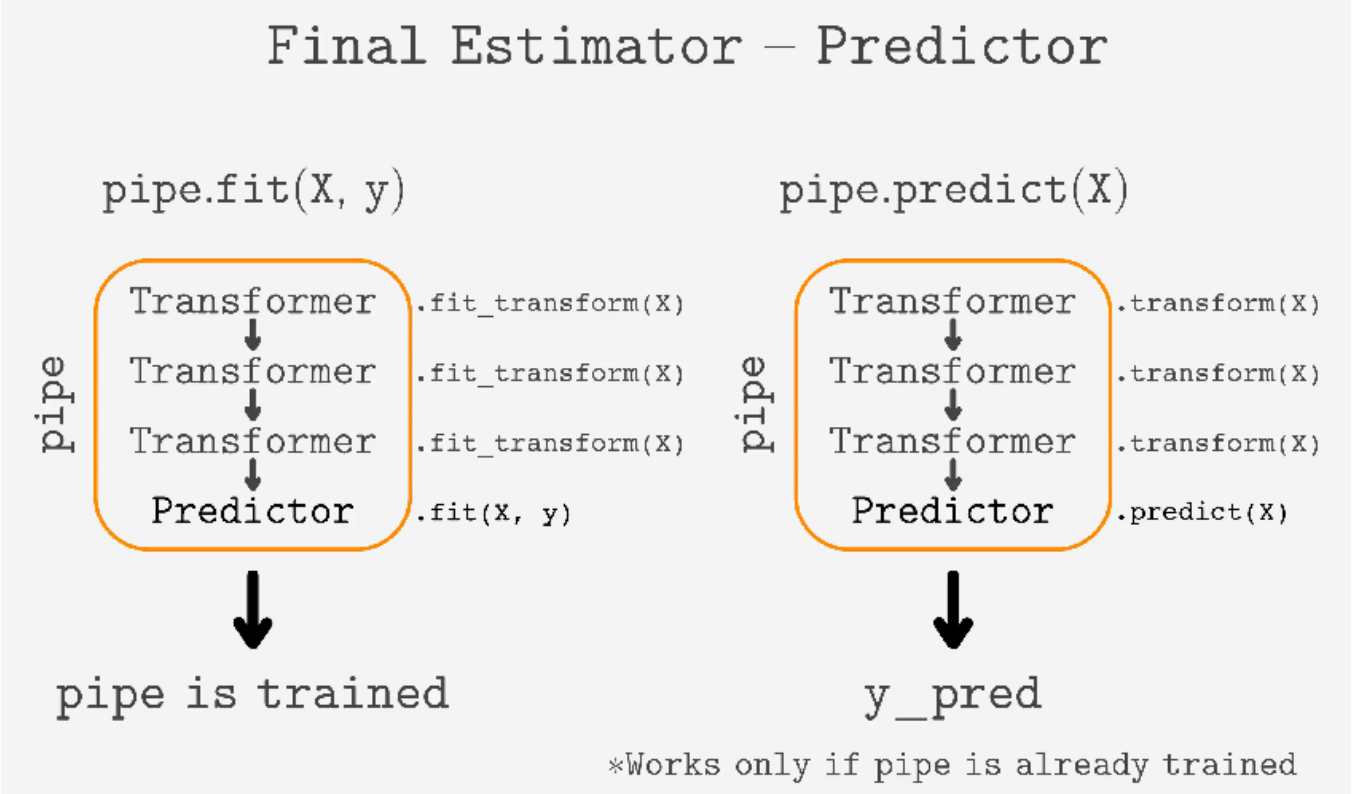
```python
import pandas as pd
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline

df = pd.read_csv('exams.csv')
# Making a column transformer
edu_categories = ['high school', 'some high school', 'some college', "associate's degree",
"bachelor's degree", "master's degree"]
ct = make_column_transformer(
  (OrdinalEncoder(categories=[edu_categories]), ['parental level of education']),
  (OneHotEncoder(), ['gender', 'race/ethnicity', 'lunch', 'test preparation course']),
  remainder='passthrough'
)
# Making a Pipeline
pipe = make_pipeline(ct, SimpleImputer(strategy='most_frequent'))
print(pipe.fit_transform(df))
```

```
[[ 3. 0. 1. ... 79. 78. 72.]
 [ 1. 0. 1. ... 81. 73. 71.]
 [ 1. 1. 0. ... 68. 75. 79.]
 ...
 [ 4. 1. 0. ... 54. 59. 68.]
 [ 2. 0. 1. ... 79. 75. 73.]
 [ 4. 1. 0. ... 79. 82. 89.]]
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Estimateur final

- Pipeline a été précédemment utilisé pour le **prétraitement**, mais son véritable objectif est de chaîner le prétraitement avec un **prédicteur final**. La dernière étape d'un pipeline peut être n'importe quel estimateur (généralement un modèle) qui produit des prédictions.
- Lors de l'appel de .fit(), chaque transformateur exécute .fit_transform(). Lors de l'appel de .predict(), le pipeline utilise .transform() avant d'envoyer les données à l'estimateur final. Cela est nécessaire car **les nouvelles données doivent être transformées exactement comme les données d'entraînement**.



Estimateur final

Pourquoi .transform() **?**

- Utiliser .fit_transform() sur de **nouvelles** données pourrait modifier les encodages (par exemple, dans OneHotEncoder), créant des colonnes non correspondantes et des prédictions non fiables. .transform() garantit un **prétraitement cohérent**, en ignorant les catégories inconnues et en conservant le même ordre de colonnes.

Instance	Couleur
1	Rouge
2	Bleu
3	Vert

Voici à quoi ressemblent les données d'entraînement après l'encodage one-hot :

Instance	Couleur_Rouge	Couleur_Bleu	Couleur_Vert
1	1	0	0
2	0	1	0
3	0	0	1

Estimateur final

Pourquoi .transform() **?**

Voici les nouvelles instances à prédire :

Instance	Couleur
4	Bleu
5	Rouge
6	Jaune

Si .fit_transform() était appliqué à **de nouvelles instances**, le OneHotEncoder pourrait générer des colonnes dans un ordre différent, voire en introduire de nouvelles. Cela entraînerait une transformation des nouvelles données **incohérente avec l'ensemble d'entraînement**, rendant les prédictions **non fiables**.

Instance	Couleur_Rouge	Couleur_Bleu	Couleur_Jaune
4	0	1	0
5	1	0	0
6	0	0	1

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Estimateur final

Pourquoi .transform() **?**

- Cependant, l'utilisation de .transform() garantit que les nouvelles données sont encodées **exactement comme les données d'entraînement**, en ignorant les catégories non observées lors de l'entraînement :

Instance	Couleur_Rouge	Couleur_Bleu	Couleur_Vert
4	0	1	0
5	1	0	0
6	0	0	0

Estimateur final

Ajout de l'estimateur final

- Il suffit d'ajouter le modèle comme **dernière étape** du pipeline :

```
pipe = make_pipeline(
        ct,
        SimpleImputer(strategy='most_frequent’),
        StandardScaler(),
        KNeighborsClassifier()
)
pipe.fit(X, y)
pipe.predict(X_new)
```

- Cela permet à l'ensemble du flux de travail—**prétraitement + prédiction**—de s'exécuter en une seule commande.

Modèles

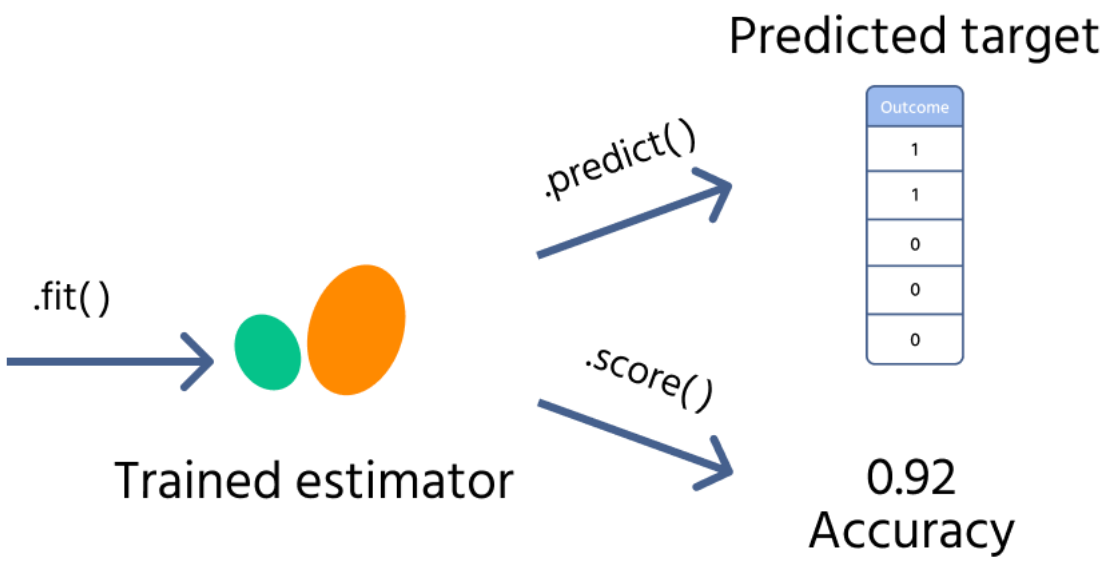
Modèles

- Un **modèle** dans Scikit-learn est un **estimateur** qui fournit les méthodes .predict() et .score(), ainsi que .fit() héritée de tous les estimateurs.

Model

(also an Estimator and Predictor)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DPF	Age	Outcome
6	148	72	35	155.5	33.6	0.627	50	1
1	85	66	29	155.5	26.6	0.351	31	0
8	183	64	29.15	155.5	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	29.15	0	25.6	0.201	30	0
3	78	50	32	88	31.0	0.248	26	1
10	115	72.41	29.15	0	35.3	0.134	29	0
...	...	...	...	...	...	...	...	...



Modèles

.fit()

- Une fois les données prétraitées et prêtes à être utilisées par le modèle, la première étape de la construction d'un modèle est **l'entraînement du modèle**. Cela se fait à l'aide de .fit(X, y).
- Pour **l'apprentissage supervisé** (régression, classification), .fit() nécessite à la fois X et y.
  Pour **l'apprentissage non supervisé** (par exemple, le clustering), il suffit d'appeler .fit(X). Passer y ne provoque pas d'erreur — il est simplement ignoré.
- Pendant l'entraînement, le modèle **apprend** les motifs nécessaires à la prédiction.

.predict()

- Après l'entraînement, utiliser .predict() pour générer des prédictions :

```
model.fit(X, y)
y_pred = model.predict(X_new)
```

.score()

- .score() évalue un modèle entraîné, généralement sur un **jeu de test** :

```
model.fit(X, y)
model.score(X_test, y_test)
```

Modèles

Exemple de code KNeighborsClassifier

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

df = pd.read_csv('penguins_pipelined.csv')
# Assign X, y variables (X is already preprocessed and y is already encoded)
X, y = df.drop('species', axis=1), df['species']
# Initialize and train a model
knn5 = KNeighborsClassifier().fit(X, y) # Trained 5 neighbors model
knn1 = KNeighborsClassifier(n_neighbors=1).fit(X, y) # Trained 1 neighbor model
# Print the scores of both models
print('5 Neighbors score:',knn5.score(X, y))
print('1 Neighbor score:',knn1.score(X, y))
```

Évaluation du Modèle

- Lors de la création d'un modèle prédictif, il est essentiel de déterminer **son niveau de performance** avant de l'utiliser pour des prédictions réelles.
- L'**évaluation du modèle** mesure la précision des prédictions du modèle. La méthode .score() fournit cette évaluation.
- Évaluer la performance sur l'ensemble d'entraînement est trompeur, car le modèle aura tendance à mieux fonctionner sur des données déjà vues. Pour obtenir une mesure réaliste, l'évaluation doit être réalisée sur des **données non vues**.
- En termes formels, l'objectif est de créer un modèle qui **généralise bien**.
- Entraîner le modèle à l'aide de l'ensemble d'entraînement, puis l'évaluer sur l'**ensemble de test** :

```
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Évaluation du Modèle

- Pour créer une **séparation aléatoire** des données, utiliser la fonction train_test_split() du module sklearn.model_selection.
- En général, la taille de l'ensemble de test dépend du jeu de données : **25–40 %** pour les petits jeux de données, **10–30 %** pour les jeux moyens, et **moins de 10 %** pour les grands jeux de données.
- Dans cet exemple, avec seulement 342 instances (un petit jeu de données), allouer **33 %** à l'ensemble de test :

```
train_test_split(X,y,test_size)

Arguments:
 • X – an array of feature values
 • y – an array of target values
 • test_size – the test set's proportion

Returns:
 • X_train, X_test, y_train, y_test – arrays of the same
datatypes as X, y
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
```

Évaluation du Modèle

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

df = pd.read_csv('penguins_pipelined.csv')
# Assign X, y variables (X is already preprocessed and y is already encoded)
X, y = df.drop('species', axis=1), df['species']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
# Initialize and train a model
knn5 = KNeighborsClassifier().fit(X_train, y_train) # Trained 5 neighbors model
knn1 = KNeighborsClassifier(n_neighbors=1).fit(X_train, y_train) # Trained 1 neighbor
model
# Print the scores of both models
print('5 Neighbors score:',knn5.score(X_test, y_test))
print('1 Neighbor score:',knn1.score(X_test, y_test))
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

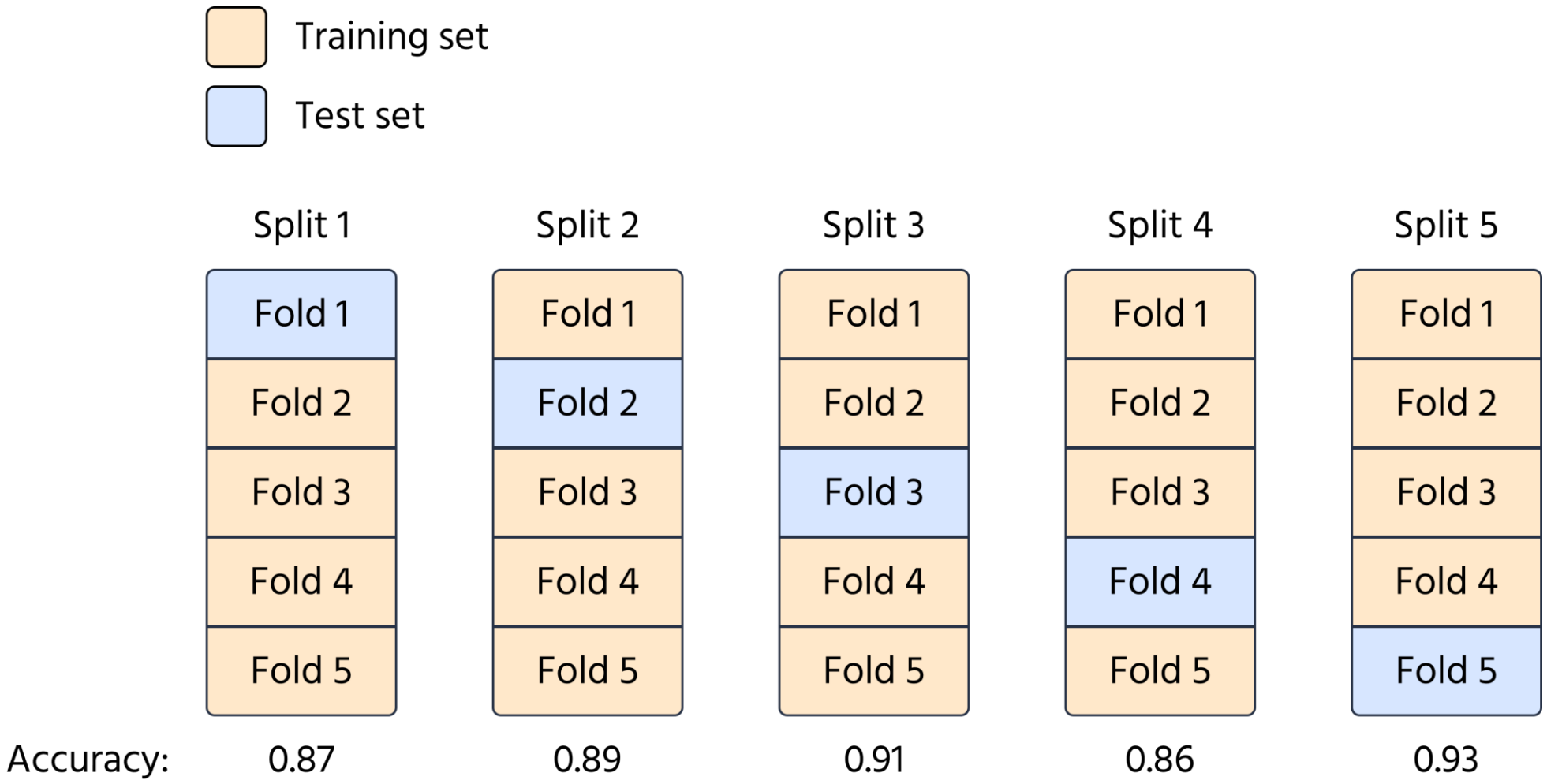
Validation Croisée

- La séparation train-test présente deux inconvénients :
  - **Moins de données d'entraînement**, ce qui peut réduire la qualité du modèle ;
  - **Dépendance à la séparation aléatoire**, entraînant des résultats instables. Pour surmonter cela, on utilise la **validation croisée**.
- La **validation croisée** est une technique pour évaluer un modèle en **le testant plusieurs fois sur des parties différentes du dataset**, afin d'obtenir une estimation plus stable et réaliste de sa performance.
- Au lieu de faire **1 seul train/test split**, on fait **K splits**, donc **K entraînements + K tests**.

Exemples :

- Tout d'abord, diviser l'ensemble du jeu de données en 5 parties égales, appelées **plis**.
- Ensuite, utiliser **un pli comme ensemble de test** et combiner les plis restants pour former l'ensemble d'entraînement.
- Comme dans tout processus d'évaluation, l'ensemble d'entraînement est utilisé pour entraîner le modèle, tandis que l'ensemble de test sert à mesurer ses performances.
- Le processus est répété afin que chaque pli serve une fois d'ensemble de test, tandis que les plis restants forment l'ensemble d'entraînement.
- La validation croisée produit plusieurs scores de précision—un par division. Leur moyenne représente la performance **moyenne** du modèle. En Python, cela se calcule avec cross_val_score().
- Vous pouvez choisir **n'importe quel nombre de plis**. Par exemple, utiliser 10 plis signifie entraîner sur 9 parties et tester sur 1. Cela se règle via le paramètre cv dans cross_val_score()

Validation Croisée



Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Validation Croisée

```
cross_val_score(estimator,X,y,scoring,cv=5)
```

Arguments:

- estimator - the model object
- X – an array of feature values
- y – an array of target values
  cv – the number of folds (5 be default)

Returns:

- scores – an array of scores at each split

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

Validation Croisée

- La validation croisée est **plus fiable** mais plus lente, car le modèle est entraîné et évalué **n fois**.
- Elle est largement utilisée dans l'**optimisation des hyperparamètres**, où la validation croisée est répétée pour chaque valeur d'hyperparamètre—par exemple, tester plusieurs valeurs de k dans k-NN. Cela permet de choisir l'option qui **offre les meilleures performances de façon constante**

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

df = pd.read_csv('penguins_pipelined.csv')
# Assign X, y variables (X is already preprocessed and y is already encoded)
X, y = df.drop('species', axis=1), df['species']
# Print the cross-val scores and the mean for KNeighborsClassifier with 5 neighbors
scores = cross_val_score(KNeighborsClassifier(), X, y)
print(scores)
print(scores.mean())
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

RechercheParGrilleCV - GridSearchCV

- Pour améliorer les performances du modèle, il convient d'ajuster les hyperparamètres. Le principe est simple : tester différentes valeurs, calculer les scores de validation croisée, puis sélectionner celle ayant le score le plus élevé.
- Ce processus peut être réalisé à l'aide de la classe GridSearchCV du module sklearn.model_selection.
- GridSearchCV (Recherche Par Grille) est une méthode qui permet de trouver automatiquement les meilleurs **hyperparamètres** d'un modèle Machine Learning

- On définit une **grille** de paramètres possibles
- Scikit-learn teste **toutes les combinaisons**
- Pour chaque combinaison, il effectue une **validation croisée (Cross-Validation)**
- Il choisit la combinaison qui donne le **meilleur score moyen**

Model	Score
KNeighborsClassifier(n_neighbors = 1)	0.83
KNeighborsClassifier(n_neighbors = 3)	0.84
KNeighborsClassifier(n_neighbors = 5)	0.86
KNeighborsClassifier(n_neighbors = 7)	0.88

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

RechercheParGrilleCV - GridSearchCV

sklearn.model_selection.GridSearchCV

Constructor:

- GridSearchCV(estimator, param_grid, scoring, cv=5):
  - estimator – the model object
  - param_grid – dictionary with parameter values to search through
  - scoring – the metric used for cross - validation score
  - cv - the number of folds (5 by default)

Methods:

- fit(X,y) – train the models using X, y
- predict(X) – predict the class for X
- score(X,y) – returns the accuracy for the X, y set

Attributes:

- best_estimator_ – object of a model with highest score
- best_score_ – the score of the best_estimator

Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

RechercheParGrilleCV - GridSearchCV

- GridSearchCV nécessite un **modèle** et une **grille de paramètres** (param_grid).
- Exemple :

```
param_grid = {'n_neighbors': [1, 3, 5, 7]}
```

- Après l'initialisation de GridSearchCV, utiliser .fit(X, y).
  - Le meilleur modèle se trouve dans .best_estimator_ ;
  - Son score de validation croisée est disponible dans .best_score_.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

df = pd.read_csv('penguins_pipelined.csv')
X, y = df.drop('species', axis=1), df['species']

param_grid = {'n_neighbors': [1,3,5,7,9]}
grid_search = GridSearchCV(KNeighborsClassifier(), param_grid)
grid_search.fit(X, y)

print(grid_search.best_estimator_)
print(grid_search.best_score_)
```

```
KNeighborsClassifier(n_neighbors=7)
0.8771952259164536
```


Pr. Ikidid Abdelouafi ~ a.ikidid@uca.ac.ma

RechercheParGrilleCV - GridSearchCV

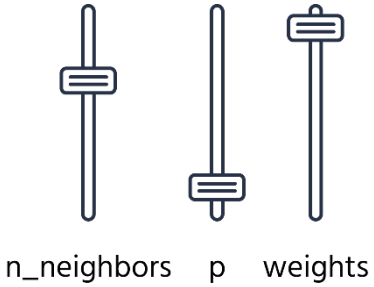
- Après l'ajustement, GridSearchCV réentraîne automatiquement le **meilleur estimateur** sur l'ensemble complet des données. L'objet grid_search devient le modèle final entraîné et peut être utilisé directement avec .predict() et .score().

```
grid_search.fit(X, y)
print(grid_search.score(X, y)) # training accuracy (not reliable for real evaluation)
```

RechercheParGrilleCV - GridSearchCV

- Avant d'utiliser GridSearchCV, il est important de noter que chaque modele possède un nombre d'hyperparamètres.
- **Exemple** : KNeighborsClassifier possède plus d'hyperparamètres que n_neighbors. Deux paramètres importants sont weights et p.
- Auparavant, seul n_neighbors était ajusté. Pour effectuer une recherche sur les trois hyperparamètres, utilisez :

```
param_grid = {
  'n_neighbors': [1, 3, 5, 7],
  'weights': ['distance', 'uniform’],
  'p': [1, 2]
}
```



- GridSearchCV essaie **toutes les combinaisons possibles** pour trouver la meilleure, il testera donc toutes celles-ci :

```
( n_neighbors=1, p=1, weights='distance' )    ( n_neighbors=5, p=1, weights='distance' )
( n_neighbors=1, p=1, weights='uniform' )     ( n_neighbors=5, p=1, weights='uniform' )
( n_neighbors=1, p=2, weights='distance' )    ( n_neighbors=5, p=2, weights='distance' )
( n_neighbors=1, p=2, weights='uniform' )     ( n_neighbors=5, p=2, weights='uniform' )
( n_neighbors=3, p=1, weights='distance' )    ( n_neighbors=7, p=1, weights='distance' )
( n_neighbors=3, p=1, weights='uniform' )     ( n_neighbors=7, p=1, weights='uniform' )
( n_neighbors=3, p=2, weights='distance' )    ( n_neighbors=7, p=2, weights='distance' )
( n_neighbors=3, p=2, weights='uniform' )     ( n_neighbors=7, p=2, weights='uniform' )
```

RechercheParGrilleCV - GridSearchCV

- Une grille plus grande comme :

```
param_grid = {
'n_neighbors': [...],
'weights': ['distance', 'uniform'],
'p': [1, 2, 3, 4, 5]
}
```

- crée 100 combinaisons. Avec une validation croisée à 5 plis, le modèle est entraîné 500 fois. Cela convient pour les petits jeux de données, mais devient trop lent pour les plus volumineux.
- Pour réduire le temps de calcul, RandomizedSearchCV teste uniquement un sous-ensemble aléatoire de combinaisons, trouvant généralement des résultats solides beaucoup plus rapidement qu'une recherche exhaustive sur grille.